ALLIN1 GUÍA DE ESTUDIO

1. Resumen

Allin1 es un marketplace compuesto por microservicios Go. Según el contexto de proyecto, IAM es responsable de autenticación y perfil de usuario; Media gestiona la carga/procesamiento de imágenes y registra metadatos de las cargas; Catalog gestiona publicaciones y sus consultas. Los servicios no deben importarse directamente entre sí: la convención del proyecto es comunicarse mediante contratos gRPC cuando un servicio necesita invocar a otro. El frontend es Next.js y es el coordinador previsto; hoy ese tramo solo se prueba con /test-media que llama a Media para subir una imagen y después a IAM para guardar la URL en el perfil.

2. Distribución entre microservicios:

El proyecto Allin1 utiliza una arquitectura de microservicios implementada en Go, en la que cada servicio tiene responsabilidades específicas.

IAM se encarga de la autenticación, la gestión de usuarios y los perfiles. En el perfil se almacenan datos personales y referencias a las imágenes del usuario, como la foto de perfil y la portada. Estas referencias se guardan como URL; el archivo de imagen no necesita almacenarse directamente en la base de datos de IAM.

Media se encarga de gestionar las imágenes. Recibe los archivos, valida su contenido, aplica el procesamiento correspondiente y los sube a Cloudinary. También registra en su propia base de datos los metadatos de cada carga, como el usuario asociado, el tipo de imagen, la URL, el identificador público de Cloudinary y la fecha de creación.

Catalog se encarga de las publicaciones y sus categorías. El contrato publication.proto contempla el campo imagenes_urls para representar las URL de imágenes asociadas a una publicación. Sin embargo, según el contexto revisado, todavía debe verificarse la persistencia de estas imágenes en el modelo y la base de datos de Catalog.

La separación permite que cada microservicio mantenga su lógica y sus datos sin depender directamente de la implementación interna de los demás. Por ejemplo, IAM puede guardar la URL de una imagen sin tener que conocer cómo Media la procesa o cómo Cloudinary la almacena.

En resumen, IAM es responsable de los datos del perfil, Media de la carga y los metadatos de las imágenes, y Catalog de la información de las publicaciones.

3. Cómo se comunican entre sí:

En Allin1, el frontend se comunica con los microservicios mediante solicitudes HTTP. Para las operaciones que requieren autenticación, envía un token JWT que permite identificar al usuario.

La carga de imágenes se realiza mediante el endpoint **POST /v1/media/upload** de Media. El frontend envía el archivo mediante **multipart/form-data**, junto con el tipo de imagen y el token JWT. Media valida la solicitud, procesa el archivo, lo sube a Cloudinary y registra los metadatos en su base de datos.

Para actualizar el perfil, el frontend utiliza el endpoint de actualización de IAM y envía la URL obtenida desde Media. IAM valida esa referencia y actualiza los campos correspondientes del perfil, como **foto_perfil_url** o **portada_url**.

Además, la convención arquitectónica del proyecto establece que los microservicios Go no deben importar directamente el código interno de otros servicios. Cuando necesitan comunicarse entre sí, deben utilizar contratos de comunicación, principalmente gRPC en la arquitectura actual.

Cloudinary se utiliza como servicio externo para almacenar y entregar las imágenes, mientras que cada microservicio mantiene su propia conexión a PostgreSQL.

gRPC Permite la comunicación entre microservicios cuando corresponde.

4. Inventario de modelos, endpoints, configuración y dependencias

Models:
IAM: Usuario

Representa a los usuarios registrados en la aplicación. Se utiliza como base para identificar al propietario de un perfil y relacionar sus datos personales con la cuenta.

IAM: Perfil

Almacena la información del perfil del usuario. Entre sus campos se encuentran foto_perfil_url y portada_url, que contienen las referencias a las imágenes utilizadas en el perfil.

Media: Media

Registra los metadatos de las imágenes que se suben al sistema. Según el esquema descrito, incluye id_media, id_usuario, tipo, url, public_id y fecha_creacion. El campo tipo permite distinguir usos como PERFIL y PORTADA.

Catalog: Publicación

Representa una publicación del marketplace. El contrato publication.proto incluye el campo imagenes_urls, pero debe verificarse si el modelo GORM y la base de datos ya permiten persistirlo.

Catalog: Categoría

Representa las categorías utilizadas para clasificar publicaciones y facilitar su consulta.

Cada servicio mantiene sus propios modelos y su persistencia. Las referencias a un usuario o a una imagen no implican que un microservicio deba importar directamente el modelo interno de otro.

5. Endpoints principales

1. POST /v1/media/upload: Microservicio Media

Permite subir una imagen de forma autenticada. Recibe una solicitud multipart con el tipo de imagen y el archivo, valida el JWT, procesa la imagen, la sube a Cloudinary y registra los metadatos en PostgreSQL. Devuelve información de la carga, incluida la URL.

2. GET /v1/auth/profile: Microservicio IAM

Obtiene el perfil del usuario autenticado. Su respuesta incluye los datos del perfil y las referencias a la foto y la portada.

3. PUT de actualización de perfil: Microservicio IAM

Permite actualizar los datos del perfil y sus referencias de imagen. La implementación descrita valida las URL de Cloudinary antes de guardarlas. Debe confirmarse la ruta HTTP exacta en el contrato o controlador.

4. GET /v1/auth/profile/{user_id}: Microservicio IAM

Obtiene un perfil público mediante su identificador. El cambio #310 descrito en el contexto incorpora una respuesta pública que no incluye el correo electrónico ni el teléfono.

5. Endpoints de publicaciones y categorías: Microservicio Catalog

Permiten consultar publicaciones, obtener una publicación específica y listar categorías. Sus contratos se encuentran en publication.proto. Para documentar cada ruta HTTP exacta, deben revisarse las anotaciones del contrato y su registro en el servidor.

Estos endpoints permiten identificar las responsabilidades de cada servicio y comprender qué componente recibe la solicitud, cuál ejecuta la lógica y dónde se guardan los datos.

Endpoint u operación	Archivo que debes revisar	Función
POST /v1/media/upload	backend-go/internal/api/media_handler.go	Recibe la solicitud HTTP y procesa los campos multipart.
Autenticación HTTP de Media	backend-go/internal/middleware/jwt_http.go	Comprueba el JWT e identifica al usuario, según la implementación descrita.
Lógica de carga	backend-go/internal/service/media/media_service.go	Coordina la carga, la validación del tipo y la persistencia de metadatos.
Procesamiento de imágenes	backend-go/internal/service/media/processor.go	Redimensiona o transforma las imágenes según las reglas implementadas.
Servicio de procesamiento y subida	backend-go/internal/service/media/upload_service.go	Coordina la lectura, validación, procesamiento y subida de archivos.

Modelo de Media	backend-go/internal/models/media.go	Define los datos que se guardan por cada imagen.
Inicialización de Media	backend-go/cmd/media/main.go	Configura y registra los componentes del servicio.
Contrato de IAM	backend-go/api/proto/auth.proto	Define las operaciones, solicitudes y respuestas de autenticación y perfil.
Lógica de perfil	Busca auth_service.go en backend-go/internal/service/	Implementa la obtención y actualización de perfiles.
Contrato de Catalog	backend-go/api/proto/publication.proto	Define operaciones, mensajes y rutas asociadas a publicaciones.
Lógica de publicaciones	Busca el archivo del servicio de publicaciones dentro de backend-go/internal/service/	Implementa consultas y operaciones sobre publicaciones.
Configuración de contenedores	docker-compose.yml	Define servicios, bases de datos, puertos y red de Docker.

Para POST /v1/media/upload, sigue este recorrido:

1. **media_handler.go**: ¿cómo recibe el archivo?
2. **jwt_http.go:** ¿cómo obtiene la identidad del usuario?
3. **media_service.go:** ¿qué validaciones y operaciones coordina?
4. **upload_service.go:** ¿cómo procesa y sube la imagen?
5. **media.go:** ¿qué datos se persisten?
6. **main.go**: ¿cómo se registra y arranca el servicio?

6. Dependencias cruzadas

Media y Cloudinary: Media depende de Cloudinary para almacenar y entregar las imágenes. El servicio necesita configurar sus credenciales y obtener la URL y el identificador público del archivo subido.

Media y PostgreSQL: Media depende de su base de datos para registrar los metadatos de las cargas. La tabla media permite relacionar cada imagen con su usuario y tipo de uso.

IAM y JWT: IAM gestiona la autenticación y la información de los usuarios. Media utiliza el JWT para comprobar la identidad del usuario que solicita una carga.

IAM y Media: IAM almacena las URL de las imágenes en el perfil, mientras Media se encarga de la carga y del procesamiento. La URL funciona como referencia entre ambos componentes; no requiere compartir el archivo ni importar directamente los modelos internos.

Catalog y las imágenes: el contrato de publicaciones incluye el campo **imagenes_urls**, pero la persistencia debe verificarse en el modelo y en la base de datos. No se debe asumir que el flujo de imágenes de publicaciones está completamente implementado.

Docker Compose y PostgreSQL: Docker Compose configura bases de datos separadas para IAM, Catalog y Media. Esto refuerza la separación de datos entre los servicios.

La arquitectura busca limitar las dependencias directas entre microservicios. La integración debe realizarse mediante interfaces y contratos definidos, evitando que un servicio dependa de la implementación interna de otro.

7. Problema identificado

Para cambiar la foto de perfil o la portada se necesitan dos operaciones en servicios distintos: primero subir la imagen a Media y luego guardar la URL devuelta en IAM. Entre ambos servicios solo viaja esa URL, que el frontend copia de una respuesta a la otra.

Acoplamiento detectado. Como IAM recibe solo una URL, debe decidir por su cuenta si es válida. En auth_service.go, cloudinaryURLPrefix() exige que empiece con https://res.cloudinary.com/<cloud>/, tomando <cloud> de CLOUDINARY_CLOUD_NAME y, si está vacío, del valor fijo rstk3bky. En el mismo archivo, fallbackAvatarURL repite ese nombre dentro de una URL completa. Media, en cambio, usa CLOUDINARY_CLOUD_NAME para subir. En docker-compose.yml el servicio media recibe esa variable, pero iam no. Por eso IAM valida siempre contra el valor fijo. Si la cuenta de Cloudinary cambiara, Media subiría imágenes que IAM rechazaría con error 400.

Alternativa A: el frontend coordina y IAM valida por contrato (elegida).
El frontend sube la imagen a POST /v1/media/upload y luego envía la URL a PUT /v1/auth/profile. Cada servicio conserva su responsabilidad y no existe dependencia directa entre ellos. Para corregir el acoplamiento se hace un refactor acotado:

1. En auth_service.go, definir una sola función cloudinaryCloudName() que lea CLOUDINARY_CLOUD_NAME y se use tanto en el prefijo de validación como en el avatar por defecto.
2. En docker-compose.yml, pasar CLOUDINARY_CLOUD_NAME al servicio iam (y en Kubernetes al configmap de IAM, coordinando con quien administra los despliegues).
3. Probar manualmente: una URL del cloud correcto devuelve 200, una URL ajena devuelve 400, un valor vacío no modifica el perfil, y al cambiar la variable IAM sigue la nueva cuenta.

Alternativa B: comunicación directa entre servicios.
Media o IAM invocan al otro por gRPC para validar la imagen o actualizar el perfil. Esto asegura la consistencia, pero exige crear un contrato nuevo (media.proto no existe hoy), un cliente y un servidor gRPC, y deja a IAM dependiendo de que Media esté disponible en ejecución. Es funcionalidad nueva y aumenta el acoplamiento.

Comparación

- Cambios necesarios: A, un archivo de código y configuración; B, contrato, servidor y cliente nuevos.
- Dependencia entre servicios: A, ninguna directa; B, en tiempo de ejecución.
- Funcionalidad nueva: A, no; B, sí.
- Reversibilidad: A, alta (configuración); B, baja.
- Consistencia: A, eventual (puede quedar una imagen sin asociar); B, mayor.

Decisión. Alternativa A, por ser la menos invasiva, mantener la separación de datos y no ampliar funcionalidad. Esta propuesta es de estudio y no se implementa en esta tarea.

Manejo de errores y límites conocidos

- Si la subida funciona y la actualización del perfil falla, la imagen queda en Cloudinary y en la tabla media sin estar asociada a ningún perfil. Se debe informar el error y permitir reintentar con la misma URL. Limpiar imágenes huérfanas es otra tarea.
- Si el INSERT en media falla después de subir, la imagen queda sin registro; hoy solo se guarda su public_id en el log.
- IAM valida el prefijo y el largo máximo, pero no comprueba que la imagen pertenezca al usuario. Es una limitación abierta.
- El frontend todavía no ejecuta este recorrido; hoy solo se prueba con /test-media. Integrarlo es trabajo futuro, no parte de esta tarea

Alcance. No se agregan endpoints, galería de publicaciones ni promedio de reseñas.

8. Go frente a NestJS

1. Despliegue liviano. El Dockerfile compila un binario estático (CGO_ENABLED=0) y lo ejecuta sobre FROM scratch, sin sistema operativo ni intérprete en la imagen final. Con Node/NestJS normalmente se despliega el runtime y las dependencias.
2. Verificación en compilación. Go es de tipado estático: muchos errores se detectan antes de ejecutar.
3. Contratos con código generado. Los .proto generan el código gRPC y el gateway REST (internal/api/pb/), de modo que contrato e implementación no se desalinean.
4. Procesamiento de imágenes. La carga y recompresión de imágenes es trabajo de CPU. La biblioteca estándar de Go y golang.org/x/image lo resuelven sin depender de librerías nativas externas.
5. Concurrencia. Las goroutines permiten atender varias solicitudes en paralelo sin configuración adicional.
6. Estructura estricta. El proyecto usa Clean Architecture y el layout estándar de Go (cmd/, internal/, api/), con un flujo fijo: proto, modelo, servicio, controlador, main.go.
7. Pruebas mediante interfaces. Por ejemplo, CloudinaryUploader permite sustituir Cloudinary por un doble en las pruebas.

9. Commits

FICHA 1: #276

1. Identificación

- **SHA completo**: b02b30622c022b9299f3a1c549890cf72daa8655
- **Enlace:** https://github.com/AlvaroFPM/Allin1_Taller_de_integracion_III/commit/b02b30622c022b9299f3a1c549890cf72daa8655
- **Autor**: gustavovlnz gvalenzuela2023@alu.uct.cl
- **Fecha**: 2026-10-13 20:46:06 -0300
- **Mensaje**: feat(#276): agregar endpoint POST /v1/media/upload con JWT, CORS y registro en BD. El cuerpo aclara que incluye el modelo Media y el AutoMigrate de #274.

2. Archivos (6, +276 −4)

- **cmd/media/main.go**: conecta la BD (si falla, sigue sin persistencia), migra Media, arma repositorio, servicio y handler, registra la ruta con HTTPAuth y agrega CORS.
- **internal/api/media/media_handler.go (nuevo):** recibe el multipart, obtiene el usuario del contexto y traduce errores a códigos HTTP.
- **internal/middleware/jwt_http.go (nuevo):** versión HTTP del interceptor gRPC; deja user_id bajo la misma UserIDKey.
- **internal/models/media.go:** agrega el modelo Media, con tipos PERFIL y PORTADA.
- **internal/service/media/media_service.go (nuevo):** valida BD y tipo, sube, guarda la referencia.
- **docker-compose.yml**: agrega `JWT_SECRET: ${JWT_SECRET}` al servicio `media`, para validar los tokens emitidos por IAM.

3. Propósito

Antes solo existía /upload/multiple, sin autenticación ni registro. Este commit agrega un endpoint autenticado que sube una foto de perfil o portada y guarda su referencia (url, public_id) en la base propia de Media.

4. Explicación técnica

- **Entradas:** multipart con tipo y file, más Authorization: Bearer.
- **Proceso:** HTTPAuth valida el token HS256 con JWT_SECRET y rechaza user_id ≤ 0. El handler limita el cuerpo a MaxFileSize + 1 MB y exige un único archivo. Upload comprueba que haya repositorio y que el tipo exista, y recién entonces sube con la carpeta media/perfil o media/portada. Luego inserta en media.
- **Salidas:** 201 con {id, tipo, url, publicId}.
- **Errores:** 405, 401, 413, 400, 415, 503 y 500.
- **Dependencias:** Cloudinary, GORM, UploadService (#169, #171, #172), middleware.UserIDKey, JWT_SECRET.
- **Decisiones:** validar antes de subir para no dejar huérfanos; id_usuario sin FK porque el usuario vive en otra base; reutilizar mapUploadError; no tocar /upload/multiple.

5. Verificación

- Pruebas: el commit no agrega tests Go. Las pruebas fueron manuales (401, 400, 415, 201 y 503 sin BD).
- Verificado en el código: el orden de validaciones y los códigos de error.
- Limitaciones: si falla el INSERT queda la imagen en Cloudinary y solo se registra el public_id en el log (el borrado es #173). ErrMediaPersist responde 500 con el prefijo «Error al procesar archivos». La lógica JWT está duplicada con el interceptor gRPC, y ambos usan el mismo secreto por defecto.

FICHA 2: #302

1. Identificación

- **SHA completo:** ecf092762a0a6a6f6be18bdabe47ef3927812a0e
- **Enlace:** https://github.com/AlvaroFPM/Allin1_Taller_de_integracion_III/commit/ecf092762a0a6a6f6be18bdabe47ef3927812a0e
- **Autor:** gustavovlnz gvalenzuela2023@alu.uct.cl ·
- **Fecha**: 2026-10-19 00:18:45 -0300
- **Mensaje**: feat(#302): limitar dimensión de imágenes según tipo PERFIL/PORTADA

2. Archivos (3, +91 –1)

- **internal/service/media/media_service.go (+7 –1):** agrega mediaMaxDimension (PERFIL 512, PORTADA 1600) y lo aplica en Upload.
- **internal/service/media/upload_service.go (+10)**: agrega WithMaxDimension.
- **frontend-marketplace/src/app/test-media/page.tsx (+74)**: página temporal para probar la subida y ver las dimensiones reales.

3. Propósito

Con #171 todas las imágenes se limitaban a 1920 px. Una foto de perfil no necesita eso: el límite por tipo ahorra peso sin tocar el procesador.

4. Explicación técnica

- **Entradas**: tipo ya validado y archivo.
- **Proceso**: Upload consulta el límite del tipo y llama a WithMaxDimension, que hace una **copia** del servicio con una copia del procesador cuyo maxDimension cambia. Luego corre ProcessFiles igual que antes.
- **Salidas**: imagen JPEG con el lado largo ≤ 512 o 1600 px, sin recortar ni ampliar.
- **Errores**: los mismos que antes.
- **Dependencias**: el campo no exportado ImageProcessor.maxDimension (#171), accesible porque es el mismo paquete.
- **Decisiones**: copiar en vez de modificar, porque UploadService se crea una sola vez en main.go y se comparte entre solicitudes concurrentes; cambiar el procesador compartido haría que una subida de PORTADA alterara el límite de una de PERFIL. Es el mismo patrón que WithFolder.

5. Verificación

- **Pruebas**: sin tests Go nuevos. Pruebas manuales con /test-media: PNG 4000×3000 como PERFIL → 512×384; como PORTADA → 1600×1200; verticales correctas; las pequeñas no se agrandan; 400, 415 y 401 correctos.
- **Verificado en el código:** la copia superficial del servicio y del procesador, y el uso en Upload.
- **Limitaciones**: si un tipo existiera en mediaFolders y faltara en mediaMaxDimension, el valor sería 0 y resizeIfNeeded reduciría la imagen a 1×1 px. **Esto lo deduje leyendo el código, no lo ejecuté.** Una mejora acotada es ignorar valores ≤ 0 en WithMaxDimension. Además, un PNG con transparencia pierde el alfa, y /test-media queda accesible en los despliegues.

FICHA 3: #307

1. Identificación

- **SHA completo:** 78f02170ecc9183b2b015f781568b52368d75a36
- **Enlace:** https://github.com/AlvaroFPM/Allin1_Taller_de_integracion_III/commit/78f02170ecc9183b2b015f781568b52368d75a36
- **Autor:** gustavovlnz gvalenzuela2023@alu.uct.cl ·
- **Mensaje:** feat(#307): agregar portada y foto de perfil editables al perfil de usuario
- **PR:** #309 (merge a3a8017, 2026-10-21, por Álvaro)

2. Archivos (6, +92 −14)

- **api/proto/auth.proto:** portada_url = 12 en la respuesta; foto_perfil_url = 6 y portada_url = 7 en el request.
- **internal/api/pb/auth/auth.pb.go:** código **generado** por protoc; no se edita a mano.
- **api/openapi/auth.config.yaml y auth.swagger.json:** documentación Swagger de los cambios.
- **internal/models/perfil.go:** foto_perfil_url pasa de 255 a 500 caracteres y se agrega portada_url.
- **internal/service/auth_service.go (+36)**: devuelve la portada, valida las URL y las guarda.

3. Propósito

El perfil no tenía portada, y la foto no se podía actualizar desde el servicio. El commit permite guardar ambas como URL de Cloudinary, que es lo que devuelve Media.

4. Explicación técnica

- **Entradas**: UpdateProfileRequest con Bearer y, opcionalmente, foto_perfil_url y portada_url.
- **Proceso**: validateUpdateProfileRequest llama a validateImageURL, que acepta vacío («no cambiar») o una URL que empiece con https://res.cloudinary.com/<cloud>/ y tenga como máximo 500 caracteres. updateProfileByUserID asigna foto y portada al crear el perfil; al actualizar, solo si vienen con valor.
- **Salidas**: ProfileResponse actualizado, o 400, 401 o 404.
- **Dependencias**: CLOUDINARY_CLOUD_NAME (con valor de respaldo rstk3bky), la tabla perfiles y el interceptor JWT.
- **Decisiones**: los campos de texto siguen con reemplazo total; los de imagen vacíos no borran; por eso no se puede borrar la portada una vez guardada; el cloud name no se trata como secreto.

5. Verificación

- **Pruebas**: el commit no agrega tests Go. Manual: GET con portadaUrl, PUT válido 200, URL ajena 400, vacío no borra. \d perfiles confirmó las columnas en varchar(500).
- **Verificado en el código:** la lógica de validateImageURL y la asignación condicional.
- **Limitaciones**: validateImageURL solo comprueba prefijo y largo, no que la imagen sea del usuario. El cloud name está repetido en fallbackAvatarURL. En Docker, IAM no recibe CLOUDINARY_CLOUD_NAME, así que siempre usa el valor de respaldo.

10. Las 8 preguntas:

1. **¿Qué hace POST /v1/media/upload y con qué códigos responde?** *(#276)*

   Sube foto de perfil o portada autenticada y responde 201 {id, tipo, url, publicId}. Errores: 405, 401, 413, 400, 415, 503 y 500. *Evidencia:* media_handler.go y mapUploadError.

2. **¿Cómo sabe Media quién es el usuario y quién creó el token?** *(#276)*

   HTTPAuth verifica HS256 con **JWT_SECRET**, lee user_id y lo guarda con UserIDKey. El token lo emite IAM en generateJWT. *Evidencia:* jwt_http.go, auth_service.go.

3. **¿Por qué Upload válida repositorio y tipo antes de subir?** *(#276)*

   Para no dejar imágenes huérfanas en Cloudinary si el registro no se puede guardar. *Evidencia:* orden de las primeras líneas de Upload.

4. **¿Qué hace WithMaxDimension y por qué devuelve una copia?** *(#302)*

   Crea un servicio con un procesador de otro límite. Copia para no modificar el servicio compartido entre solicitudes simultáneas. *Evidencia:* upload_service.go, y que main.go crea un solo uploadService.

5. **¿Qué pasaría si falta el límite de un tipo en mediaMaxDimension?** *(#302)*

Valdría 0 y, según el código, la imagen se reduciría a 1×1 px. Hoy no ocurre porque ambos mapas tienen los mismos tipos. Se evita ignorando valores ≤ 0.

6. **¿Por qué el valor vacío significa “no cambiar” y qué no garantiza la validación?** *(#307)*

   Evita que un PUT sin imágenes borre las guardadas; el costo es que no se puede quitar la portada. Solo comprueba prefijo y largo, no que la imagen sea del usuario. *Evidencia:* validateImageURL, if foto != "".

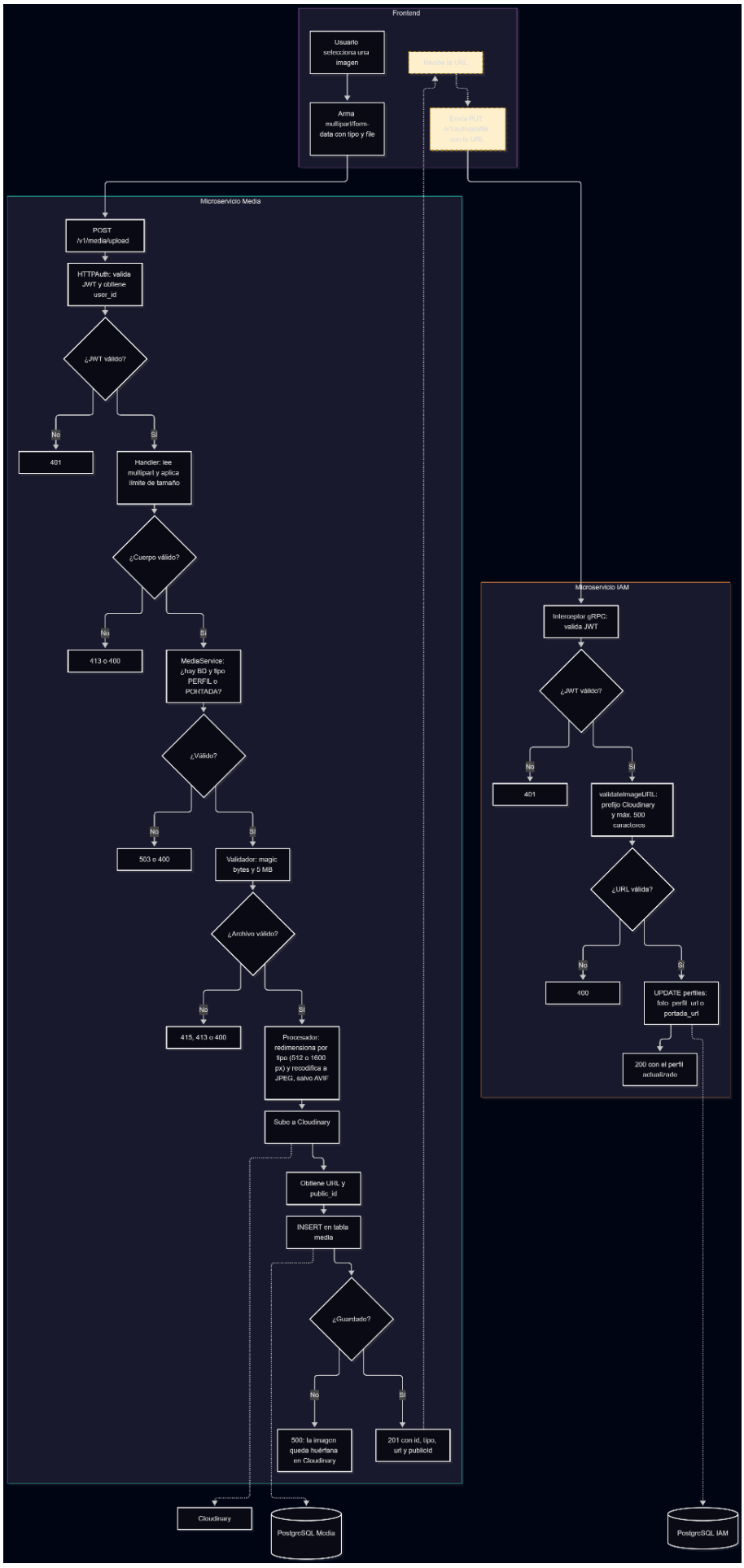
7. **¿Qué efecto tiene #307 en la base de datos y en el despliegue?** *(#307)*

   IAM migra Perfil al arrancar: agrega portada_url y amplía foto_perfil_url a 500. El redeploy aplica ese cambio. *Evidencia:* cmd/iam/main.go (AutoMigrate) y \d perfiles.

8. **Si el INSERT falla después de subir, ¿qué queda y cómo lo mejorarías?** *(#276)*

   Queda la imagen en Cloudinary sin fila en media; hoy solo se loguea el public_id y el borrado está pendiente (#173). Mejoras: borrar al fallar o limpiar huérfanos periódicamente. También pueden quedar filas de media que nunca llegaron a un perfil. *Evidencia:* rama de error de Upload.

Diagrama de Flujo:


Frontend
Usuario selecciona una imagen
Arma multipart/form-data con tipo y file
Recibe la URL
Envía PUT /v1/auth/profile con la URL
Microservicio Media
POST /v1/media/upload
HTTPAuth: valida JWT y obtiene user_id
¿JWT válido?
No
401
Sí
Handler: lee multipart y aplica límite de tamaño
¿Cuerpo válido?
No
413 o 400
Sí
MediaService: ¿hay BD y tipo PERFIL o PORTADA?
¿Válido?
No
503 o 400
Sí
Validador: magic bytes y 5 MB
¿Archivo válido?
No
415, 413 o 400
Sí
Procesador: redimensiona por tipo (512 o 1600 px) y recodifica a JPEG, salvo AVIF
Sube a Cloudinary
Obtiene URL y public_id
INSERT en labla media
¿Guardado?
No
500: la imagen queda huérfana en Cloudinary
Sí
201 con id, tipo, url y publicId
Microservicio IAM
Interceptor gRPC: valida JWT
¿JWT válido?
No
401
Sí
validateImageURL: prefijo Cloudinary y máx. 500 caracteres
¿URL válida?
No
400
Sí
UPDATE perfiles: foto_perfil_url o portada_url
200 con el perfil actualizado
Cloudinary
PostgreSQL Media
PostgreSQL IAM

8. Preguntas abiertas y límites de lo que se puede afirmar

- Verificar el diff y la autoría exacta de los tres commits seleccionados; completar SHA completo y enlaces.

- Confirmar el estado final de PR/merge y resultados de go test para #302, #307 y #310; el contexto los marca como pendientes de confirmación.
- Revisar el código NestJS histórico antes de describir cómo resolvía carga, perfiles o referencias. El material disponible no detalla su comportamiento.
- El endpoint /upload/multiple aparece descrito como no autenticado y sin persistencia: documentar como deuda/riesgo, no como flujo equivalente al endpoint autenticado.
- Confirmar si la página /test-media se mantiene accesible en el despliegue y si debe retirarse; el contexto la identifica como riesgo.
- La galería de fotos de publicaciones y el promedio de reseñas quedan diferidos. No indicar que están implementados ni añadirlos como trabajo de esta semana.

9. Plan de estudio de 150 minutos

35 min — Trazar el flujo: Dibujar el flujo de este documento de memoria. Verificar handler, middleware, service, processor, repositorio y modelo. Marcar qué pasos están comprobados por código y cuáles son inferencias de integración.

45 min — Inventario: Recorrer los archivos de evidencia de la sección 4. Anotar modelos, endpoints, variables de entorno, dependencias y responsabilidades de IAM/Media/Catalog. Revisar el código NestJS histórico por separado si está disponible.

45 min — Alternativas y defensa: Explicar en voz alta las alternativas A/B y defender por qué A es menos acoplada. Responder las ocho preguntas sin leer y volver al código para corregir cualquier respuesta no respaldada.

25 min — Cierre y registro: Completar los SHA/autores/enlaces; anotar dudas; exportar/subir este archivo a la tarea de estudio correspondiente. No marcar como completadas las funcionalidades diferidas.